Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра **09.02.07 Информационные системы и программирование**

ПРАКТИЧЕСКАЯ РАБОТА № 4

Дисциплина(модуль)/предмет **МДК.03.02**

Тема: **Измерение метрик качества кода**

ВАРИАНТ № **3**

Студент (ка)/учащийся: **Бойцов Даниил Антонович** / _____ /
Ф. И. О. подпись

Группа / Класс **И13-24-11**

Руководитель: **Чудинова А.А.** /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «04» ноября 2026 г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

Содержание

1 Объект измерения и правила расчёта.....3
2 Результат до и после рефакторинга.....4
3 Оптимизированный код и проверка.....5
Подтверждающие материалы.....6
Заключение.....7
Список использованных источников.....8

1 Объект измерения и правила расчёта

Цель работы: оценить сложность, поддерживаемость и зависимости контроллера, выполнить рефакторинг и повторить измерение. Сравниваются LegacyClimateController.send и RefactoredClimateController.Send. Отдельный учебный пример позволяет не смешивать архитектурное упрощение контроллера с добавлением всех защитных middleware.

CC и MI рассчитаны Microsoft.CodeAnalysis.Metrics 4.14.0 через CodeAnalysisMetricData. Исходники компилируются в Roslyn с reference assemblies .NET 8; результат сохранён в metrics-before.json и metrics-after.json. Метрики не сняты из вымышленного окна Visual Studio: используется реальная библиотека Microsoft [1, 2].

Цикломатическая сложность показывает число независимых путей. Для исходного send: базовый путь 1, восемь if и один логический || дают CC=10. Для Send после извлечения сервиса остаётся выбор результата через ?:, поэтому CC=2. Число тестов не обязано буквально равняться CC: важны ещё данные и взаимодействия.

```
MI = max(0, (171 - 5.2*ln(V) - 0.23*CC
        - 16.2*ln(LOC))*100/171)
```

Формула MI приведена для пояснения подхода Microsoft; фактические значения взяты из инструмента, а не получены подстановкой произвольного объёма V [3]. Class Coupling учитывается тем же инструментом. Значения разных анализаторов нельзя сравнивать без согласования правил подсчёта.

Связность дополнительно оценивается LCOM-HS по явно указанной матрице использования полей. Конструкторы исключены. Для Legacy: три метода, два поля; send использует оба, count — data, report — logs. LCOM-HS=(3−4/2)/(3−1)=0.5. Это ручной расчёт, сохранённый в cohesion.json, а не встроенная метрика Microsoft.

2 Результат до и после рефакторинга

Метрика и уровень	**До**	**После**
CC ключевого метода	10	2
MI ключевого метода	58	81
Class Coupling метода	6	5
SourceLines метода	23	6
MI класса контроллера	75	89
Class Coupling класса	8	6
CC класса с учётом членов	14	3
SourceLines класса	44	12

Сложность ключевого метода уменьшилась на 80 %, MI вырос на 23 пункта. У класса после рефакторинга есть явный конструктор, поэтому его CC включает дополнительный путь конструктора. Сравнение относится к контроллеру, а не доказывает уменьшение всей сложности системы.

Класс и набор методов	**LCOM-HS**	**Интерпретация**
Legacy: send, count, report	0.5	Контроллер объединял два состояния и несколько обязанностей.
ReadingStore: Add и Count.get	0	Оба члена работают с одной коллекцией.
DiagnosticLog: Record и Report.get	0	Оба члена работают с одной коллекцией сообщений.
RefactoredClimateController: Send	Не определён	При одном рабочем методе показатель LCOM-HS не рассчитывается.

Обнаруженные запахи: метод объединяет проверку, сохранение и диагностику; транспортный слой держит коллекции; короткие имена затрудняют чтение. После извлечения зависимость контроллера направлена на IReadingService. Правила диапазонов находятся в ReadingRules, а сообщения — в ReadingDiagnostics. Ошибка || исправлена отдельно как изменение поведения.

3 Оптимизированный код и проверка

```
public sealed class RefactoredClimateController : ControllerBase
{
  private readonly IReadingService _service;
  public RefactoredClimateController(IReadingService service)
    => _service = service;

  public IActionResult Send(string id, decimal temperature,
    decimal humidity, int sequence)
  {
    bool accepted = _service.Accept(id, temperature, humidity, sequence);
    return accepted ? Ok() : BadRequest();
  }
}

public bool Accept(string id, decimal temperature,
  decimal humidity, int sequence)
{
  if (!ReadingRules.IsValid(id, temperature, humidity, sequence))
    return false;
  var reading = new Reading(id, temperature, humidity, sequence);
  _store.Add(reading);
  ReadingDiagnostics.Record(reading, _log);
  return true;
}

public static bool IsValid(string id, decimal t, decimal h, int seq)
  => !string.IsNullOrEmpty(id) && seq >= 1
   && t >= -40 && t <= 85 && h >= 0 && h <= 100;
```

Листинг объединяет фрагменты контроллера, сервиса и класса правил; полные классы находятся в samples/Climate.After. Сервис не скрывает проблему одним переименованием: границы диапазона явно соединены &&. Четыре регрессионных случая R01 проверяют допустимые границы, температуру 100 и влажность 250; все пройдены.

Небольшой контроллер проще читать и подменять в тестах, однако его MI не подтверждает защищённость сети. Для рабочего API HMAC, политика доступа, FluentValidation и ограничение частоты проверяются в ПР8 независимо от результата метрик.

Подтверждающие материалы

Рисунок 1 дополняет материалы работы. На снимке приведены измеренные показатели сравниваемых методов. Цикломатическая сложность снизилась с 10 до 2, индекс сопровождаемости вырос с 58 до 81.

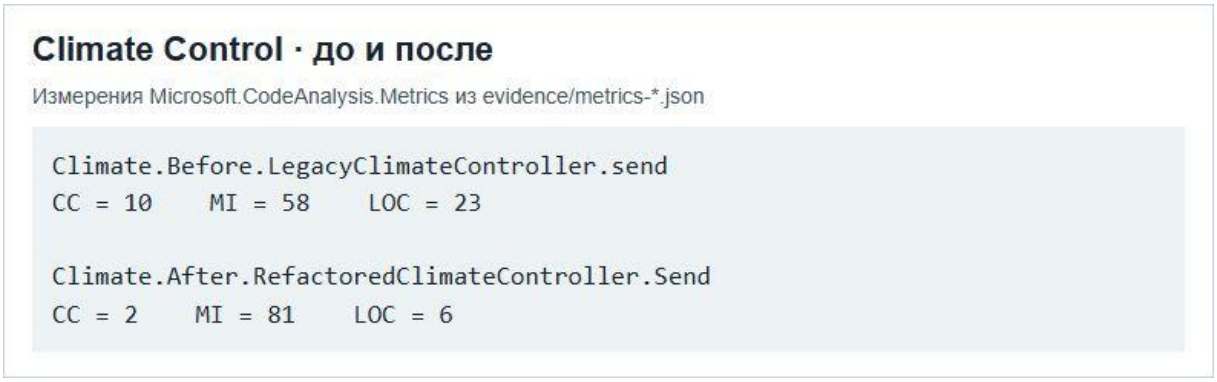


Рисунок 1 — Метрики исходного и обновлённого методов

Материалы работы на GitHub: открыть раздел работы

Заключение

Метрики получены из реального анализа исходников: CC метода 10→2, MI 58→81, сцепление класса 8→6. Рефакторинг разделил обязанности и сопровождается регрессионными тестами; при интерпретации сохранены границы применимости каждого показателя.

Материалы относятся к модулю Climate Control варианта 3. Проверенный программный стенд не подключён к физическому отоплению или вентиляции кампуса. Промышленная приёмка оборудования в рамках работы не проводилась.

Список использованных источников

1. Microsoft Learn. Code metrics values. URL: https://learn.microsoft.com/en-us/visualstudio/code-quality/code-metrics-values.

2. Microsoft Learn. Generate code metrics data. URL: https://learn.microsoft.com/en-us/visualstudio/code-quality/how-to-generate-code-metrics-data.

3. Microsoft Learn. Maintainability index range and meaning. URL: https://learn.microsoft.com/en-us/visualstudio/code-quality/code-metrics-maintainability-index-range-and-meaning.